ProgressProbe: Weakly-Supervised Temporal Progress Estimation for LLM Agent Trajectories

Anonymous

May 14, 2026

Abstract

LLM-based agents frequently exhibit *semantic stagnation* during long-horizon tasks—executing superficially diverse but collectively ineffective action sequences that waste computational budget without advancing toward the goal. Detecting such stagnation is difficult: actions are non-repetitive at the surface level, and step-wise scoring methods lose the temporal dynamics of progress within a window of steps. We propose PROGRESSPROBE, a framework that combines (1) a *data-programming* weak-supervision pipeline that transfers the Snorkel paradigm to sequential agent trajectories, producing window-level progress labels from three domain-specific labeling functions, and (2) a lightweight GRU probe with attention that learns temporal progress signals from these noisy labels. On SWE-bench agent trajectories, ProgressProbe achieves 0.91 macro-F1 on majority-vote labels and 0.86 macro-F1 on a 400-window human-annotated gold set, outperforming the best heuristic baseline (Repeat-3) by +6.24 pp and +6.89 pp respectively. The probe surpasses even the labeling-function ensemble oracle (+4.43 pp) and a few-shot Claude Sonnet 4 judge (+3.74 pp) while being 606× faster and 31× cheaper. A zero-shot cross-agent transfer from SWE-agent to OpenHands retains 0.83 macro-F1, confirming that the learned signal captures general goal-directed progress rather than agent-specific behavioral patterns. Simulated early termination driven by the probe saves 32.7% of tokens while preserving 96.1% of task success rate.

1 Introduction

LLM-powered autonomous agents [Yang et al.(2024), Wang et al.(2024)] increasingly tackle complex, multi-step tasks such as software engineering, data analysis, and web navigation. A persistent failure mode in these systems is *semantic stagnation*: the agent continues to act—issuing diverse commands, editing files, reading documentation—yet makes no meaningful progress toward its goal. Unlike simple infinite loops or exact command repetition, semantic stagnation is characterized by surface-level diversity that masks underlying ineffectiveness, making it invisible to cycle detectors and structurally repetitive action filters.

Existing approaches to monitoring agent progress fall into three categories, none of which fully address the problem. *Cycle detectors* [Chen et al.(2024a), Guo et al.(2024)] identify structural repetition but miss semantically diverse stagnation. *Step-level value estimators* [Luo et al.(2024), Chen et al.(2024b)] score individual actions and aggregate uniformly, losing the temporal signature of stagnation onset within a window. *Self-reflection mechanisms* [Zhang et al.(2024)] rely on the stagnating agent itself to diagnose the problem—a fundamentally unreliable signal.

We reframe progress stagnation detection as a *window-level progress estimation* problem and make two contributions:

A data-programming framework for agent trajectories. We design three labeling functions (LFs)—`file_diff_delta`, `command_repetition`, and `outcome_backpropagation`—that capture complementary dimensions of progress and stagnation. Their majority vote produces window-level labels that achieve 84.5% accuracy against human annotations (Fleiss $\kappa = 0.724$). This constitutes a non-trivial

transfer of the Snorkel paradigm [Ratner et al.(2017)] to sequential trajectory data, where LFs exhibit temporal dynamics (leading vs. lagging indicators) absent in i.i.d. settings.

Empirical evidence that temporal modeling matters. A lightweight GRU probe with attention, trained on these weakly-supervised labels, achieves 0.91 macro-F1—outperforming the LF ensemble oracle by 4.43 pp, demonstrating that the probe learns richer representations beyond the rules that generated its training signal. The 13.72 pp gap over mean-pooled embeddings with logistic regression confirms that temporal modeling of step-to-step transitions, not merely better features, drives the improvement.

The architecture is a deliberate engineering choice, not a contribution: our novelty lies in the weak-supervision methodology and the systematic empirical study across training regimes, baselines, and agent implementations.

2 Related Work

Agent monitoring and intervention. Recent work on detecting failure modes in LLM agents spans cycle detection [Chen et al.(2024a)], trajectory anomaly detection [Guo et al.(2024)], and process reward models [Luo et al.(2024), Chen et al.(2024b)]. These approaches either target structural repetition (missing semantic stagnation) or operate at step granularity (losing window-level temporal dynamics). ProgressProbe operates at the window level, capturing stagnation onset patterns that step-level methods average away.

Data programming and weak supervision. The Snorkel framework [Ratner et al.(2017)] introduced labeling functions for programmatic data annotation in NLP. Extensions have been applied to relation extraction, sentiment analysis, and medical NLP [Ratner et al.(2019)]. Our work is, to our knowledge, the first to apply data programming to sequential agent trajectory data, where the temporal ordering of LF activations introduces dynamics not present in the original i.i.d. formulation.

Probing learned representations. Probing classifiers are widely used to analyze what information is encoded in neural representations [Belinkov(2022)]. We adopt the probing methodology not for analysis but for *deployment*: the probe is a lightweight, real-time progress estimator trained on frozen embeddings.

3 Method

3.1 Problem Formulation

Given an agent trajectory $\tau = (a_1, a_2, \ldots, a_T)$ where each a_t is an action (command execution, file edit, etc.), we define a *sliding window* $w_i = (a_i, a_{i+1}, \ldots, a_{i+k-1})$ of k consecutive steps. The task is binary classification: assign each window a label $y \in \{\texttt{progress}, \texttt{stagnation}\}$. We use $k = 5$ by default (see Section 6 for ablation).

3.2 Weak Supervision via Labeling Functions

We design three labeling functions, each capturing a distinct dimension of progress:

`file_diff_delta`. Measures the magnitude of file changes across the window. If the cumulative diff size decreases or remains zero across consecutive steps, the LF votes `stagnation`. This is the strongest individual signal (Section 6.3), directly measuring output change.

`command_repetition`. Detects whether the agent repeats semantically equivalent commands. Beyond exact string matching, it uses embedding similarity to identify paraphrased commands. This LF is the earliest leading indicator of stagnation, firing on average 1.8 steps before the other LFs reach consensus (Table 10).

`outcome_backpropagation`. Propagates the final trajectory outcome backward: if the trajectory ultimately fails and the window shows no corrective action, it votes `stagnation`. This is a lagging indicator (average lag: 2.1 steps) that provides complementary signal from the future.

Label aggregation. We aggregate LF votes via majority vote. Windows where at least 2 of 3 LFs agree (conf$\geq$2) form a high-confidence subset; the remaining conf=1 windows have only a single LF vote and higher noise (25–30% error rate).

3.3 Embedding Extraction

Each action a_t is embedded using a frozen sentence encoder to obtain $\mathbf{e}_t \in \mathbb{R}^{1024}$. A window w_i is represented as a sequence $(\mathbf{e}_i, \ldots, \mathbf{e}_{i+k-1})$.

3.4 GRU Probe with Attention

The probe processes the embedding sequence through a single-layer bidirectional GRU:

$$\mathbf{h}_t = \text{BiGRU}(\mathbf{e}_t, \mathbf{h}_{t-1}) \tag{1}$$

An attention mechanism computes a weighted summary:

$$\alpha_t = \frac{\exp(\mathbf{v}^\top \tanh(\mathbf{W}\mathbf{h}_t))}{\sum_{j=1}^{k} \exp(\mathbf{v}^\top \tanh(\mathbf{W}\mathbf{h}_j))} \tag{2}$$

$$\mathbf{c} = \sum_{t=1}^{k} \alpha_t \mathbf{h}_t \tag{3}$$

The context vector $\mathbf{c}$ is passed through a linear classifier. The model has 1.32M parameters and trains in $\sim$12 minutes.

3.5 Training Objective

We train with cross-entropy loss augmented by a supervised contrastive (SupCon) auxiliary loss with weight $\lambda = 0.3$ and temperature $\tau = 0.07$:

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda \cdot \mathcal{L}_{\text{SupCon}} \tag{4}$$

SupCon encourages same-class hidden states to cluster and different-class states to separate, improving embedding separation from 0.761 to 0.847 (Table 14).

4 Experimental Setup

Data. We extract trajectories from SWE-bench Verified runs of SWE-agent, yielding 54,809 windows (conf$\geq$1) of which 15,401 are high-confidence (conf$\geq$2). For cross-agent evaluation, we process 2,195 OpenHands instances producing 48,352 windows.

Human-annotated gold set. Three annotators independently label 400 windows (156 stagnation, 244 progress). Inter-annotator agreement is Fleiss $\kappa = 0.724$ (substantial); 68.2% of windows achieve unanimous agreement. Majority-vote labels match human gold labels with 84.5% accuracy.

Method	F1	F1-S	F1-P	ΔR3
GRU Probe	**.862**	**.809**	**.916**	+6.89
Claude S4 (fs)	.841	.783	.899	+4.78
LF Oracle	.819	.751	.886	+2.53
Repeat-3	.793	.720	.867	—
Mean-pool+LR	.729	.623	.834	−6.45

Table 1: Results on 400-window human-annotated gold set. F1 = macro-F1; F1-S/F1-P = stagnation/progress F1; ΔR3 = gap vs. Repeat-3 in pp.

Metric	Value
Fleiss κ	0.724
3/3 Agreement	68.2%
2/3 Agreement	31.8%
MV Accuracy vs Gold	84.5%
MV Errors (stag→prog)	38/400
MV Errors (prog→stag)	24/400

Table 2: Majority-vote label quality against human gold set.

Baselines. We compare against: (1) **Repeat-3**: heuristic detecting $\geq$3 consecutive identical actions; (2) **Cosine Similarity**: pairwise embedding cosine between consecutive steps; (3) **LF Oracle**: direct majority vote of LF outputs at test time; (4) **Mean-pool + LR/SVM**: mean-pooled window embeddings with logistic regression or RBF-SVM; (5) **Max-pool + LR**: max-pooled embeddings with LR; (6) **LLM-as-Judge**: Claude Sonnet 4 and GPT-4o in zero-shot and few-shot settings; (7) **Non-NLP (GBT)**: gradient-boosted trees on 18 hand-crafted structural features.

Metrics. We report macro-F1, per-class F1 for stagnation and progress, and coverage.

5 Results

5.1 Main Results on Human Gold Set

Table 1 presents results on the 400-window human-annotated gold set, providing evaluation free of circular dependency with the weak-supervision labels.

The probe achieves 0.862 macro-F1 on human labels, outperforming all baselines. The +6.89 pp gap over Repeat-3 and +4.36 pp over the LF oracle confirm that the probe learns signal beyond its training labels. The LLM judge (Claude Sonnet 4, few-shot) achieves 0.841 but at 31$\times$ the cost (Section 5.3).

Label quality validation. Table 2 validates the weak-supervision labels. The majority-vote accuracy of 84.5% against human gold labels, with Fleiss $\kappa = 0.724$, confirms that the data-programming pipeline produces reliable training signal despite operating with only three LFs.

5.2 Probe vs. Baselines on Majority-Vote Labels

Table 3 compares all methods on majority-vote labels across confidence subsets.

The probe trained on full conf$\geq$1 data (54,809 windows) achieves 0.874 macro-F1, only 3.13 pp below the conf$\geq$2-trained probe (0.906) despite 72% of training windows having noisy single-LF labels. Both settings outperform Repeat-3 by $>$6 pp, confirming noise tolerance. The probe exceeds the LF oracle by 4.43 pp on conf$\geq$2 and 5.39 pp on full data, demonstrating that temporal modeling synthesizes richer signal than the rules that generated the training labels.

Method	F1 (c≥2)	F1 (full)	Cov.
GRU Probe	**.906**	**.874**	100%
LF Oracle	.861	.820	71.9%
Repeat-3	.843	.819	100%
Cosine Sim.	.313	—	100%

Table 3: Probe vs. baselines on majority-vote labels. c≥2: high-confidence subset; full: all conf≥1 windows.

Method	F1	F1-S	$/1K	Lat.
GRU Probe	**.906**	**.859**	$0.41	4.7ms
Claude S4 (fs)	.868	.810	$12.84	2.85s
Claude S4 (zs)	.835	.776	$11.20	2.41s
GPT-4o (fs)	.853	.795	$9.63	3.12s
GPT-4o (zs)	.819	.759	$8.47	2.78s

Table 4: Probe vs. LLM-as-judge. F1-S = stagnation F1; $/1K = cost per 1,000 windows; Lat. = latency per window. Evaluated on 500-window sample.

5.3 LLM-as-Judge Comparison

Table 4 shows that even the best LLM judge (Claude Sonnet 4, few-shot) underperforms the probe by 3.74 pp while costing 31.3× more and running 606× slower. LLMs are particularly weak on stagnation detection (0.810 vs. 0.859 F1-S): they tend to classify windows with active command execution as progress even when the commands are collectively ineffective. The probe's advantage comes from domain-calibrated decision boundaries learned from 15K+ training examples, versus the LLM's reliance on general reasoning about what constitutes progress.

5.4 Temporal vs. Static Aggregation

Table 5 isolates the effect of temporal modeling. Mean-pooling discards step ordering and achieves only 0.768 macro-F1—13.72 pp below the GRU. The gap is not due to classifier capacity (LR on 1024-dim features has ample expressiveness) but to the loss of sequential dependencies. Stagnation F1 drops most severely (0.657 vs. 0.859) because stagnation is defined by a *transition* from effective to ineffective actions—a temporal pattern that mean-pooling averages away. Notably, even the simple Repeat-3 heuristic (0.843) outperforms mean-pooling (0.768), since it captures at least one temporal pattern (consecutive repetition).

5.5 Cross-Agent Transfer

Table 6 evaluates whether the probe captures general progress signals or agent-specific behaviors. A probe trained exclusively on SWE-agent trajectories achieves 0.835 macro-F1 when applied zero-shot to 48,352 OpenHands windows (2,195 instances), a 7.08 pp drop from within-agent performance. The drop stems from distribution shift in action formats: OpenHands uses structured actions (e.g., `CmdRunAction`, `FileEditAction`) while SWE-agent uses free-form bash commands. Despite this shift, the probe outperforms Repeat-3 on OpenHands by 4.53 pp and the LF oracle (which requires format adaptation) by 3.66 pp. The LF oracle barely exceeds Repeat-3 (+0.87 pp) because the LFs were designed for SWE-agent's format and lose precision after adaptation—yet the probe generalizes through the embedding space, confirming that it has learned transferable progress signals.

Method	Agg.	F1	F1-S	ΔGRU
GRU+Att	Seq.	**.906**	**.859**	—
Mean+LR	Mean	.768	.657	−13.72
Mean+SVM	Mean	.782	.674	−12.34
Max+LR	Max	.741	.630	−16.43
Repeat-3	Rule	.843	.751	−6.24

Table 5: Temporal vs. static aggregation. Seq. = learned sequential processing. The gap confirms temporal modeling is the key factor.

Setting	Agent	F1	F1-S	F1-P	ΔR3
Within	SWE-agent	.906	.859	.952	+6.24
Cross (0-shot)	OpenHands	.835	.782	.887	+4.53
Repeat-3	OpenHands	.789	.716	.863	—
LF Oracle	OpenHands	.798	.723	.873	+0.87

Table 6: Cross-agent transfer. The SWE-agent-trained probe retains 0.835 macro-F1 on OpenHands trajectories without fine-tuning.

5.6 Downstream Application: Early Termination

We simulate probe-driven early termination on held-out trajectories (Table 7). When the probe's stagnation probability exceeds a threshold for consecutive windows, the agent is terminated. At the optimal threshold (0.6), 32.7% of tokens are saved while preserving 96.1% of task success. We observe an *intervention paradox*: 17.3% of terminated trajectories would have eventually succeeded in the original run, but only after consuming an average of 2,847 additional tokens on ineffective actions—making early termination cost-effective even accounting for these cases.

5.7 Error Analysis

Tables 8 and 9 analyze the probe's 221 errors (7.2% error rate). False positives (98 cases) concentrate on exploratory behavior—the agent reads files, runs searches, and gathers information without producing edits, which the probe interprets as stagnation. Of these, 67% occur in trajectories that ultimately succeed, suggesting that the probe is overly pessimistic about information-gathering phases. False negatives (123 cases) arise from semantic drift: the agent tries diverse approaches (different fix strategies, alternative commands) that all fail, but their surface diversity fools the probe into predicting progress. Compared to Repeat-3, the probe uniquely corrects 187 cases (semantically diverse stagnation that Repeat-3 misses) while making only 34 errors where Repeat-3 is correct—a net gain of 153 windows.

5.8 LF Temporal Dynamics

A distinctive property of applying data programming to sequential data is that LFs exhibit temporal dynamics (Table 10). `command_repetition` is the earliest indicator, detecting behavioral repetition 1.8 steps before the other LFs converge. `file_diff_delta` fires near-simultaneously with stagnation onset (+0.3 steps). `outcome_backpropagation` is a lagging indicator (−2.1 steps), providing retrospective confirmation. At stagnation onset, LF agreement is only 34%, rising to 71% at peak stagnation. This explains the higher noise in conf=1 windows: they predominantly capture onset phases where LFs have not yet converged, and the probe must learn to classify these ambiguous early signals.

Threshold	Tokens Saved	Success Pres.	False Term.
0.5	38.4%	94.2%	5.8%
0.6 (optimal)	32.7%	96.1%	3.9%
0.7	27.1%	97.8%	2.2%
0.9	14.3%	99.4%	0.6%

Table 7: Early termination tradeoffs. At the optimal threshold (0.6), the probe saves 32.7% of tokens while preserving 96.1% of task success rate.

Error Type	Count	Dominant Pattern
FP (prog→stag)	98	Exploratory search w/o edits
FN (stag→prog)	123	Diverse but ineffective cmds

Table 8: Error analysis on conf$\geq$2 test set (221 total errors out of 3,080 windows).

6 Ablation Studies

6.1 Window Size

k=7 yields the highest F1 (0.915) but introduces 2 extra steps of detection latency compared to k=5 for only +0.93 pp. k=3 degrades by 3.31 pp because 3 steps provide insufficient context to distinguish exploratory behavior from stagnation.

6.2 Architecture

The core finding is that temporal modeling matters far more than the specific architecture: GRU+Attention and Transformer differ by only 0.88 pp, while both exceed the non-sequential MLP by $>$12 pp. The Transformer uses 2$\times$ the parameters but underperforms slightly, likely because 5-step sequences are too short for self-attention to provide benefit over recurrence. Attention adds 1.64 pp over GRU+Mean-pool, providing interpretable stagnation-onset localization as a bonus.

6.3 LF Hold-One-Out

Each LF contributes non-redundantly (Table 13). `file_diff_delta` has the largest marginal contribution ($-$4.21 pp when removed) and the largest coverage drop (to 58.3%), consistent with file changes being the most direct progress indicator in SWE tasks. `outcome_backpropagation` contributes least ($-$1.52 pp), as its retrospective signal overlaps most with windows already covered by the other two LFs.

6.4 SupCon Auxiliary Loss

SupCon provides a modest but consistent +1.31 pp gain (Table 14), primarily improving the separation of borderline windows near the stagnation onset boundary.

6.5 NLP vs. Non-NLP Features

Table 15 justifies the centrality of natural language understanding. A gradient-boosted tree classifier on 18 structural features (line counts, file counts, command diversity, edit entropy) achieves only 0.723 macro-F1—18.21 pp below the probe. Structural features capture surface patterns (e.g., "zero lines changed $\Rightarrow$ stagnation") but cannot distinguish semantically effective from semantically ineffective actions when both produce file modifications. This result anchors our contribution within the NLP community: understanding the *content* of agent actions, not merely their structural footprint, is essential for progress estimation.

Category	Count	% of Errors
Both wrong	87	39.4%
Probe wrong, R3 correct	34	15.4%
Probe correct, R3 wrong	187	—
Both correct	2,772	—

Table 9: Error overlap between probe and Repeat-3. The probe uniquely corrects 187 cases where Repeat-3 fails while losing only 34.

LF	Role	Lead/Lag	First-fire
`cmd_rep`	Leading	+1.8 steps	52%
`file_diff`	Coincident	+0.3 steps	31%
`outcome_bp`	Lagging	−2.1 steps	17%

Table 10: LF temporal dynamics at stagnation onset. LF agreement is only 34% at onset but rises to 71% at stagnation peak.

6.6 Data Scaling

The probe reaches 0.823 macro-F1 with only 10% of the data (1,540 windows), demonstrating the framework's data efficiency (Table 16). Performance saturates around 60% of training data, after which marginal gains diminish (50%→100% yields only +1.79 pp). This suggests that for new domains or agents, a small number of trajectories processed through the LF pipeline suffices to train an effective probe.

7 Analysis

Interpretability. Visualization of GRU hidden states via t-SNE reveals clear separation between progress and stagnation clusters (silhouette score: 0.412). Stagnation windows form three sub-clusters corresponding to repetitive, drifting, and static stagnation patterns—consistent with the error analysis typology. Attention weights concentrate on steps 2–3 in stagnation windows (concentration coefficient: 0.73), indicating that the model learns to identify the *onset* of stagnation. Progress windows show roughly uniform attention, consistent with each step contributing to forward movement.

The intervention paradox. 17.3% of trajectories flagged for termination would have eventually succeeded. However, these "late recoveries" consumed an average of 2,847 additional tokens before resolving—making their token cost far exceed the savings from early termination. This creates a principled basis for threshold selection: the optimal threshold (0.6) minimizes cost-adjusted expected value rather than simply trading accuracy for coverage.

8 Discussion

Limitations. Our evaluation covers two coding agents (SWE-agent, OpenHands) on SWE-bench. While the cross-agent transfer results are encouraging, we have not validated on non-coding agents (web navigation, data analysis) where the semantics of "progress" may differ substantially. The LFs are domain-specific by design; applying ProgressProbe to a new domain requires designing new LFs, though the framework and probe architecture transfer directly.

Progress as a spectrum. Our binary framing (progress vs. stagnation) simplifies what is inherently a continuous spectrum. The error analysis reveals that information-gathering phases—genuine progress

k	#Win	F1	F1-S	Latency
3	72,841	.872	.814	3 steps
5	54,809	**.906**	.859	5 steps
7	43,267	.915	**.871**	7 steps

Table 11: Window size ablation. k=5 balances accuracy and detection latency.

Architecture	F1	F1-S	Params	Time
GRU+Attention	**.906**	**.859**	1.32M	12m
Transformer (2L)	.897	.848	2.67M	28m
GRU+Mean-pool	.889	.839	1.31M	11m
MLP on Concat	.781	.669	0.53M	5m

Table 12: Architecture ablation. Both sequential architectures (GRU, Transformer) substantially outperform the MLP, while differing $<$1 pp from each other.

toward understanding, not toward output—are systematically misclassified. Future work could adopt a three-class scheme (progress / exploration / stagnation) or continuous progress scoring.

From Snorkel to sequences. Applying data programming to sequential data surfaces properties absent in the i.i.d. setting. LFs become temporal entities with leading/lagging dynamics, and their agreement rate varies systematically with stagnation phase. This suggests that temporal label models—extensions of Snorkel's generative model that account for LF ordering—could further improve label quality.

9 Conclusion

We presented ProgressProbe, a weakly-supervised framework for detecting semantic stagnation in LLM agent trajectories. Three contributions emerge from systematic experimentation. First, a data-programming pipeline with three labeling functions produces training labels that achieve 84.5% accuracy against human annotations, enabling probe training without manual labeling. Second, a lightweight GRU probe trained on these labels outperforms the labeling functions themselves (+4.43 pp over LF oracle), the best heuristic (+6.24 pp over Repeat-3), and frontier LLM judges (+3.74 pp over Claude Sonnet 4 few-shot) at 606$\times$ lower latency. Third, the probe transfers zero-shot across agent implementations (SWE-agent $\rightarrow$ OpenHands, 0.835 F1), confirming that it captures general progress signals. Deployed as an early-termination trigger, ProgressProbe saves 32.7% of tokens while preserving 96.1% of task success—a practical path to more efficient agent execution.

References

[Yang et al.(2024)] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.

[Wang et al.(2024)] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. OpenDevin: An open platform for AI software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.

[Chen et al.(2024a)] Yangruibo Chen, Marcus J. Min, Saurabh Sinha, and Baishakhi Ray. On the effectiveness of code agent monitoring and self-repair. *arXiv preprint*, 2024.

[Guo et al.(2024)] Zhixuan Guo, Changzhi Sun, and Wanxiang Che. SpecRA: Speculation with early rejection for efficient LLM agent interactions. *arXiv preprint*, 2024.

LF Config	#LFs	Cov.	F1	Δ
All 3 LFs	3	71.9%	.906	—
− `file_diff`	2	58.3%	.863	−4.21
− `cmd_rep`	2	64.1%	.881	−2.43
− `outcome_bp`	2	69.5%	.890	−1.52

Table 13: LF hold-one-out ablation. Each LF provides non-redundant signal.

Setting	F1	Embed Sep.	Δ
CE + SupCon	.906	.847	—
CE only	.892	.761	−1.31

Table 14: SupCon ablation. SupCon improves embedding separation and F1.

[Luo et al.(2024)] Zhiyu Luo, Wei Shen, and others. AgentPRM: Process reward models for LLM agent planning. *arXiv preprint*, 2024.

[Chen et al.(2024b)] Yuntao Chen and others. SWE-TRACE: Trajectory-level credit assignment for software engineering agents. *arXiv preprint*, 2024.

[Zhang et al.(2024)] Shishir Zhang, Jun Araki, and others. Agent collaboration for self-reflection and planning. *arXiv preprint*, 2024.

[Ratner et al.(2017)] Alexander Ratner, Stephen H. Bach, Henry Ehrenberg, Jason Fries, Sen Wu, and Christopher Ré. Snorkel: Rapid training data creation with weak supervision. In *Proceedings of the VLDB Endowment*, 11(3):269–282, 2017.

[Ratner et al.(2019)] Alexander Ratner, Braden Hancock, Jared Dunnmon, Frederic Sala, Shreyash Pandey, and Christopher Ré. Training complex models with multi-task weak supervision. In *Proceedings of AAAI*, 2019.

[Belinkov(2022)] Yonatan Belinkov. Probing classifiers: Promises, shortcomings, and advances. *Computational Linguistics*, 48(1):207–219, 2022.

Method	Features	F1	F1-S
GRU Probe	NL Embed (1024d)	**.906**	**.859**
Mean+LR	NL Embed (1024d)	.768	.657
Non-NLP (GBT)	Structural (18d)	.723	.601
Random	—	.496	—

Table 15: NLP vs. non-NLP features. Natural language embeddings are necessary: structural features alone achieve only 0.723 macro-F1.

Train %	Size	F1	Δ vs 100%
10%	1,540	.823	-8.21
25%	3,850	.861	-4.43
50%	7,700	.888	-1.79
100%	15,401	.906	—

Table 16: Data scaling. Performance saturates around 60% of training data ($\sim$9,240 windows), demonstrating the framework's data efficiency.